Théorie des graphes

Chapitre 3: Problème de plus court chemin

asma.ghdami@esprit.tn

September 13, 2022

Outline

Le problème de plus court chemin

Condition Nécessaire

Principe d'optimalité

Algorithmes de résolution

- Algorithme de Dijkstra
- Algorithme de Ford-Bellman
- Algorithme de Floyd

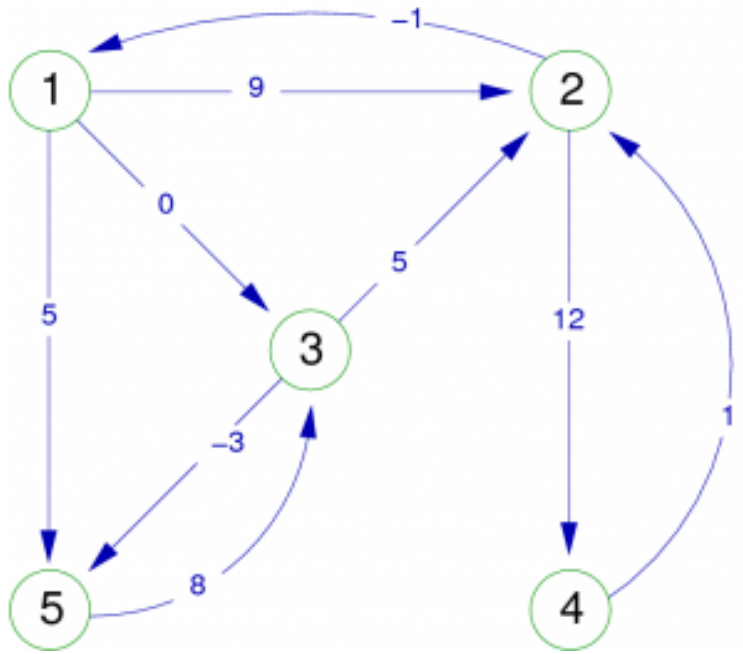
Le problème de plus court chemin

Le problème de la recherche de plus court chemin dans un graphe se rencontre dans de nombreuses applications. On peut citer entre autres:

- les problèmes d'optimisation de réseaux (routiers, télécommunications),
- le calcul du coût de transport,
- le calcul du temps de parcours,
- certaines méthodes de traitement numérique du signal,
- certains problèmes d'investissement et de gestion de stocks.

Graphes valués

Etant donné un graphe orienté $G = (V, E)$, on associe à chaque arc $u \in E$ un nombre $l(u) \in \mathbb{R}$ appelé "longueur" de l'arc. On dit alors que G est valué par les longueurs $l(u)$. Si $u = (i, j)$, on utilisera également la notation C_{ij} pour la longueur de l'arc u.



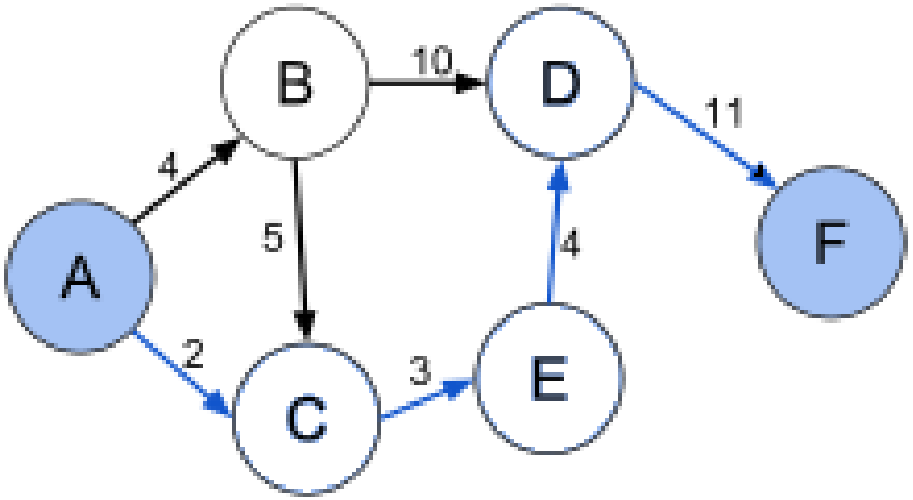
Problème de plus court chemin

Définition: Problème de plus court chemin

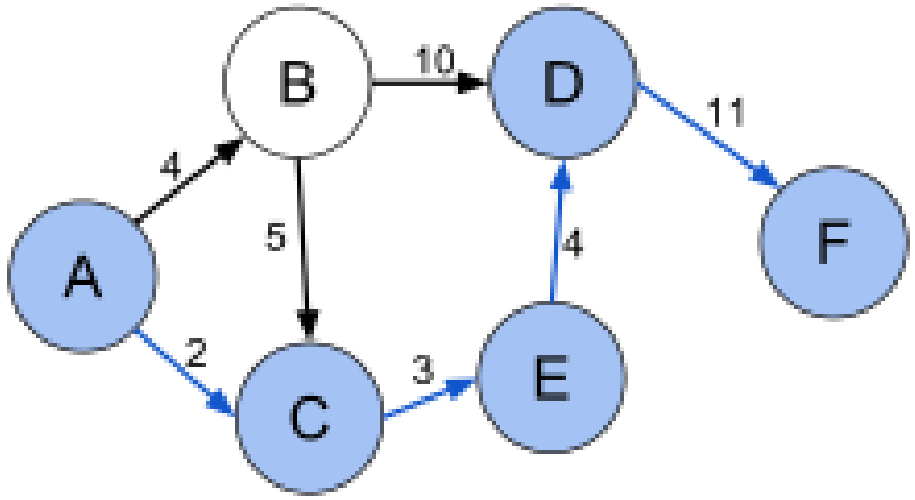
Le problème de plus court chemin (PCC) entre deux sommets *i* et *j* du graphe consiste à déterminer, parmi tous les chemins allant de *i* à *j*, un chemin noté μ^*, dont la longueur totale:

$$l(\mu^*) = \sum_{u} l(u) \quad \text{soit minimale.}$$

Exemple:



Exemple du plus court chemin du sommet A au sommet F ($A - C - E - D - F$)



Les algorithmes seront différents suivant les propriétés des graphes:

- $l(u) \geq 0, \forall u \in V$
- $l(u)$ quelconques
- Graphe sans circuit

Et suivant le problème considéré:

- Recherche du plus court chemin d'un sommet à tous les autres
- Recherche du plus court chemin entre tous les couples de sommets.

Content

Le problème du plus court chemin

Condition Nécessaire

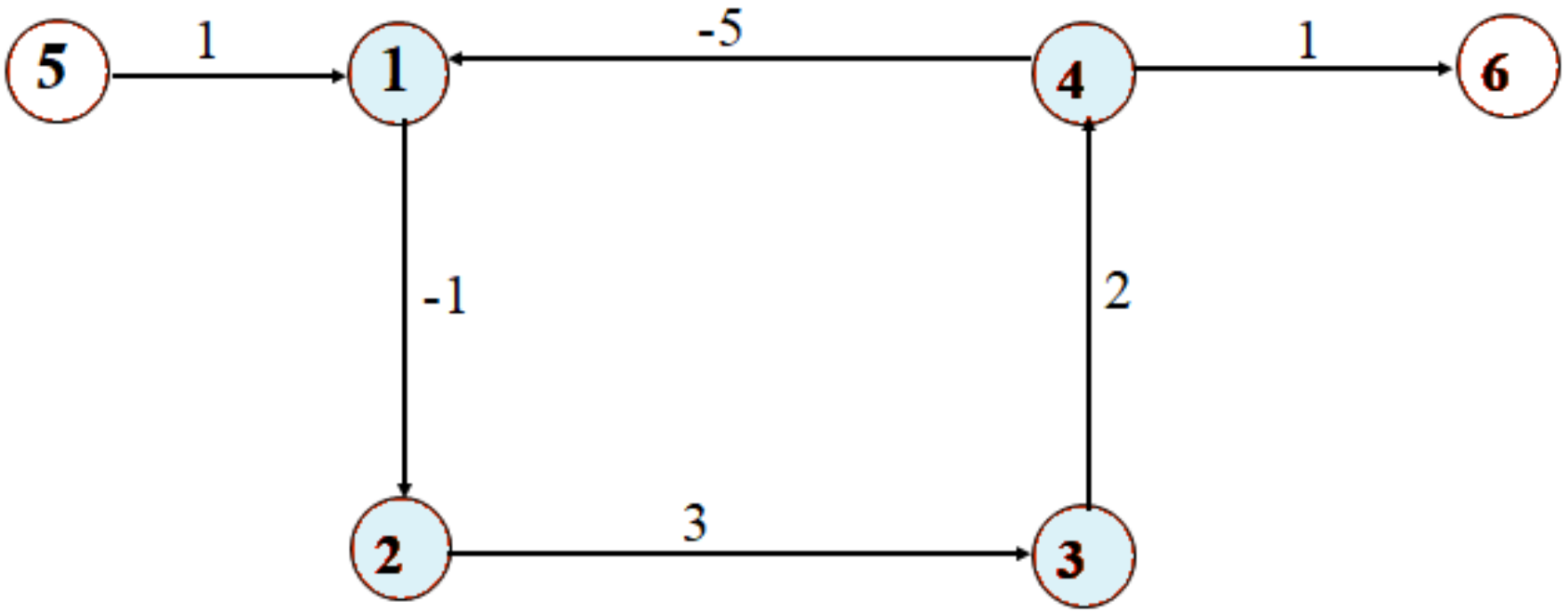
Principe d'optimalité

Algorithmes de résolution

- Algorithme de Dijkstra
- Algorithme de Ford-Bellman
- Algorithme de Floyd

Condition Nécessaire

- Le PCC a une solution si et seulement s'il n'existe pas dans le graphe un circuit de longueur strictement négative pouvant être atteint à partir de l'origine *s*.
- Un circuit négatif est appelé circuit absorbant.



Content

Le problème du plus court chemin

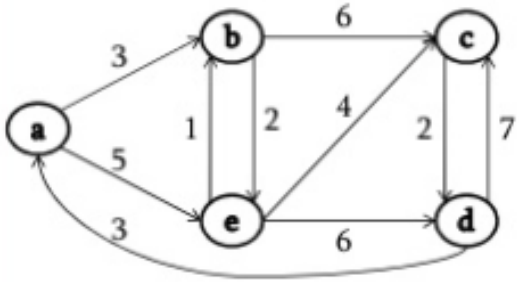
Condition Nécessaire

Principe d'optimalité

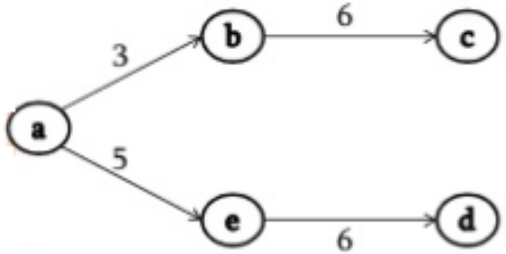
Algorithmes de résolution
- Algorithme de Dijkstra
- Algorithme de Ford-Bellman
- Algorithme de Floyd

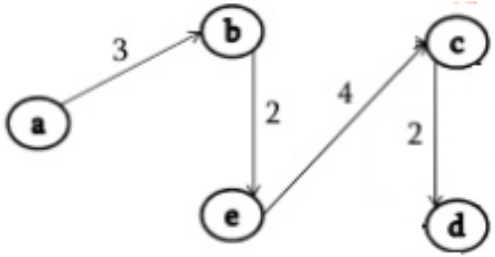
Principe d'optimalité

Tout chemin optimal est constitué de sous-chemins optimaux.



Exemples des plus courts chemins dont l'origine est a





Content

Le problème du plus court chemin

Condition Nécessaire

Principe d'optimalité

Algorithmes de résolution
- Algorithme de Dijkstra
- Algorithme de Ford-Bellman
- Algorithme de Floyd

Algorithmes de résolution

Selon les propriétés du graphe traité (avec/sans circuit ou longueurs positives/quelconques) et selon le problème considéré (recherche de plus court chemin d'un sommet vers tous les autres ou entre tous les couples de sommets), il existe de nombreux algorithmes permettant l'obtention d'une solution.

Algorithme de Dijkstra

- L'algorithme porte le nom de son inventeur, l'informaticien néerlandais Edsger Dijkstra, et a été publié en 1959.
- L'algorithme de Dijkstra sert à résoudre le problème du plus court chemin.
- Plus précisément, il calcule des plus courts chemins à partir d'une source vers tous les autres sommets dans un graphe orienté pondéré par des réels positifs.

Algorithme de Dijkstra

- $d(j)$ est la longueur du plus court chemin reliant 1 à j. Les $d(j)$ sont initialisées à certaines valeurs puis seront mises à jour au fur et à mesure tout le long de l'algorithme.
- Les sommets du graphe sont partitionnés en deux sous ensembles: P et T.
  Avec : P : c'est l'ensemble des sommets (j) dont les $d(j)$ sont fixées d'une façon Permanente.
  T : c'est l'ensemble des sommets (j) dont les $d(j)$ sont fixées d'une façon Temporaire.

Pseudo-code

- Etape 0: Initialisation

  $d(1) = 0$

  Faire: $\forall j \in \{2, ..., n\}$

  $$\text{d(j)} = \begin{cases} C_{1,j} & \text{si l'arc(1,j)} \in \text{graphe} \\ \infty & \text{si non} \end{cases}$$

  $PCC(j) = (1, j)$, le plus court chemin qui relie les sommets 1 et j.

  Fin. $P = \{1\}$ et $T = \{2, ..., n\}$.

- Etape 1: Choix d'un $d(k)$ optimal

  Déterminer le noeud k tq. $d(k) = \min_{j \in T} d(j)$

  Faire: $P = P \cup \{k\}$ et $T = T / \{k\}$

  Fin.

  Si $T = \emptyset$, alors Fin

- Etape 2: Mise à jour des $d(j)$

  Faire: $\forall (j) \in T$ et successeur de (k)

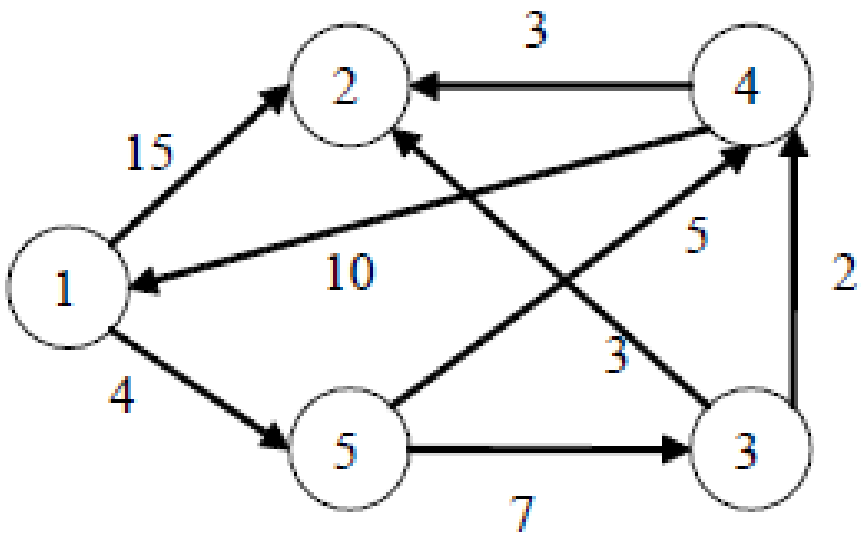
  Si $d(k) + C_{kj} < d(j)$, alors $d(j) = d(k) + C_{kj}$

  $$Pcc(j) = PCC(k) \cup (k, j)$$

  Fin.

  Aller à l'étape (1).

Exemple 1.



Le plus court chemin entre le sommet (1) et le reste des sommets:

	It(0)	
	$d(j)$	$Pcc(j)$
2	15	$(1,2)$
3	∞	$(1,3)$
4	∞	$(1,4)$
5	4	$(1,5)$
P	$\{1\}$	

Le plus court chemin entre le sommet (1) et le reste des sommets:

	It(0)		It(1)	
	$d(j)$	$Pcc(j)$	$d(j)$	$Pcc(j)$
2	15	$(1,2)$	15	$(1,2)$
3	∞	$(1,3)$	11	$(1,5,3)$
4	∞	$(1,4)$	9	$(1,5,4)$
5	4	$(1,5)$	-	-
P	$\{1\}$		$\{1,5\}$	

Le plus court chemin entre le sommet (1) et le reste des sommets:

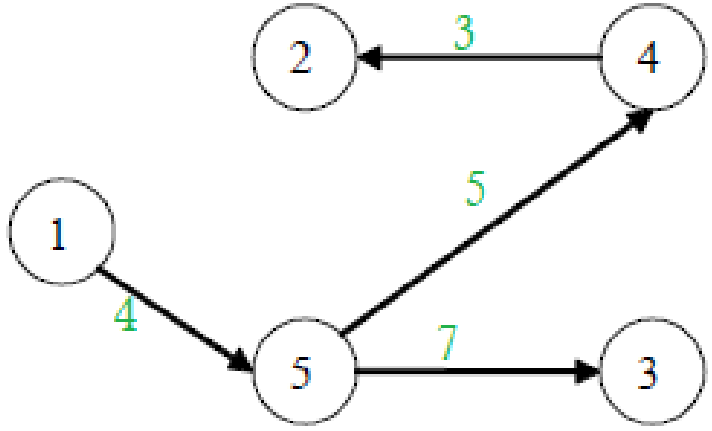
	It(0)		It(1)		It(2)	
	$d(j)$	$Pcc(j)$	$d(j)$	$Pcc(j)$	$d(j)$	$Pcc(j)$
2	15	$(1,2)$	15	$(1,2)$	12	$(1,5,4,2)$
3	∞	$(1,3)$	11	$(1,5,3)$	11	$(1,5,3)$
4	∞	$(1,4)$	9	$(1,5,4)$	-	-
5	4	$(1,5)$	-	-	-	-
P	$\{1\}$		$\{1,5\}$		$\{1,5,4\}$	

Le plus court chemin entre le sommet (1) et le reste des sommets:

	It(0)		It(1)		It(2)		It(3)	
	$d(j)$	$Pcc(j)$	$d(j)$	$Pcc(j)$	$d(j)$	$Pcc(j)$	$d(j)$	$Pcc(j)$
2	15	$(1,2)$	15	$(1,2)$	12	$(1,5,4,2)$	12	$(1,5,4,2)$
3	∞	$(1,3)$	11	$(1,5,3)$	11	$(1,5,3)$	-	-
4	∞	$(1,4)$	9	$(1,5,4)$	-	-	-	-
5	4	$(1,5)$	-	-	-	-	-	-
P	$\{1\}$		$\{1,5\}$		$\{1,5,4\}$		$\{1,5,4,3\}$	

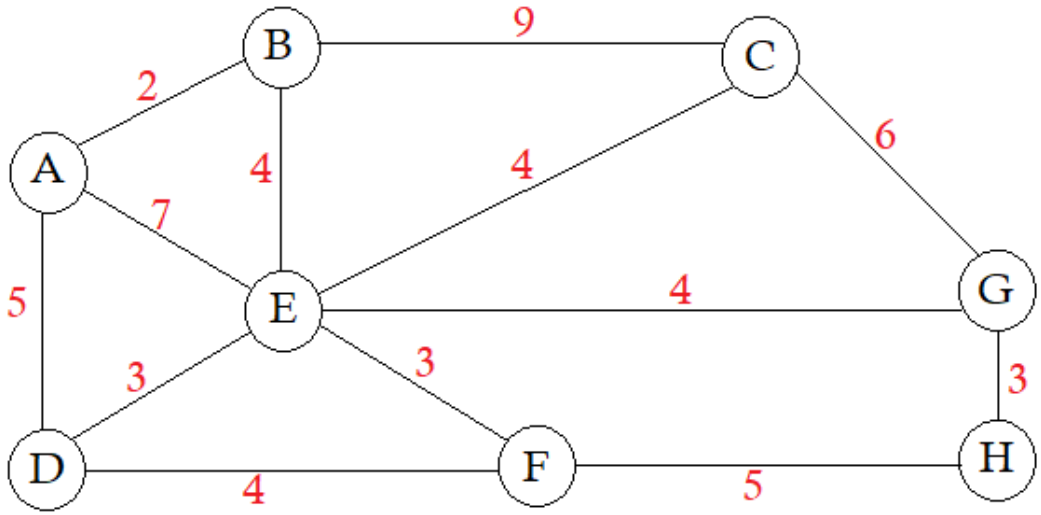
Le plus court chemin entre le sommet (1) et le reste des sommets:

	It(0)		It(1)		It(2)		It(3)		It(4)	
	$d(j)$	$Pcc(j)$	$d(j)$	$Pcc(j)$	$d(j)$	$Pcc(j)$	$d(j)$	$Pcc(j)$	$d(j)$	$Pcc(j)$
2	15	$(1,2)$	15	$(1,2)$	12	$(1,5,4,2)$	12	$(1,5,4,2)$	-	-
3	∞	$(1,3)$	11	$(1,5,3)$	11	$(1,5,3)$	-	-	-	-
4	∞	$(1,4)$	9	$(1,5,4)$	-	-	-	-	-	-
5	4	$(1,5)$	-	-	-	-	-	-	-	-
P	$\{1\}$		$\{1,5\}$		$\{1,5,4\}$		$\{1,5,4,3\}$		$\{1,5,4,2\}$	



Exemple 2.

Un livreur de pizzas doit livrer une commande en moins de 14 minutes. Chaque arête du graphe indique la durée, exprimée en minutes des trajets entre les différentes carrefours de son arrondissement. La société fabriquant les pizzas se situe au sommet *A* du graphe et la livraison doit s'effectuer au sommet *H*.



1. Pour quelles raisons le livreur peut résoudre ce problème en utilisant l'algorithme de Dijkstra?
2. Le livreur arrivera-t-il au temps?

1. Il s'agit d'un graphe qui représente un réseau routier (ou carte routière) dans lequel chaque arête représente une route à double sens. Dans ce cas chaque arête sera dédoublée en 2 arcs de sens opposé et de même longueur.

Application de l'algorithme de Dijkstra:

	It(0)		It(1)		It(2)		It(3)		It(4)		It(5)		It(5)	
	$d(j)$	$Pcc(j)$	$d(j)$	$Pcc(j)$	$d(j)$	$Pcc(j)$	$d(j)$	$Pcc(j)$	$d(j)$	$Pcc(j)$	$d(j)$	$Pcc(j)$	$d(j)$	$Pcc(j)$
B	2	(A,B)	-	-	-	-	-	-	-	-	-	-	-	-
C	∞	(A,C)	11	(A,B,C)	11	(A,B,C)	10	(A,B,E,C)	10	(A,B,E,C)	10	(A,B,E,C)	-	-
D	5	(A,D)	5	(A,D)	-	-	-	-	-	-	-	-	-	-
E	7	(A,E)	6	(A,B,E)	6	(A,B,E)	-	-	-	-	-	-	-	-
F	∞	(A,F)	∞	(A,F)	9	(A,D,F)	9	(A,D,F)	-	-	-	-	-	-
G	∞	(A,G)	∞	(A,G)	∞	(A,G)	10	(A,B,E,G)	10	(A,B,E,G)				
H	∞	(A,H)	∞	(A,H)	∞	(A,H)	∞	(A,H)	14	(A,B,E,F,H)	13	(A,B,E,G,H)	13	(A,B,E,G,H)
P	{A}		{A,B}		{A, B, D}		{A, B, D, E}		{A, B, D, E, F}		{A, B, D, E, F, G}		{A, B, D, E, F, G, C}	

Algorithme de Ford-Bellman

Algorithme de Ford-Bellman

- L'algorithme de Ford-Bellman, est un algorithme qui calcule des plus courts chemins depuis un sommet source donné dans un graphe orienté pondéré.
- Contrairement à l'algorithme de Dijkstra, l'algorithme de Ford-Bellman autorise la présence des arcs de poids négatif.

Pseudo-code

- Etape 0: Initialisation

  $d^1(1) = 0$

  Faire: $\forall j \in \{2, ..., n\}$

  $$\mathrm{d}^1(j) = \begin{cases} C_{1,j} & \text{si l'arc(1,j)} \in \text{graphe} \\ \infty & \text{si non} \end{cases}$$

Fin.

$k = 1$.

- Etape 1: Mise à jour des $d^k(j)$

  Faire: $\forall(j)$

  $$d^{k+1}(j) = \min\left(d^k(j), \min\left(d^k(l) + C_{lj}\right)\right)$$

  avec, (l) prédécesseur de (j).

  Fin.

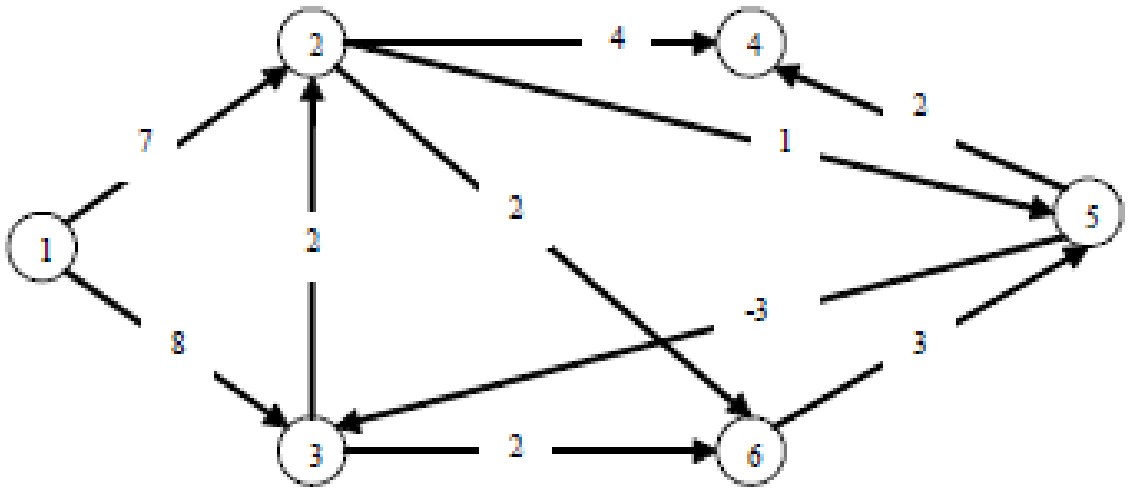
- Etape 2: Test de convergence

  Si $d^{k+1}(j) = d^k(j)$, $\forall j$, alors Fin (optimalité).

  Sinon et si $k = n - 1$, alors il existe un circuit de longueur négative, sinon optimalité.

  Sinon faire $k = k + 1$ et aller à l'étape (1).

Exemple 1.



	$k=1$	
	$d^1(j)$	$Pcc(j)$
1	0	$(1,1)$
2	7	$(1,2)$
3	8	$(1,3)$
4	$+\infty$	$(1,4)$
5	$+\infty$	$(1,5)$
6	$+\infty$	$(1,6)$

	$k=1$		$k=2$	
	$d^1(j)$	$Pcc(j)$	$d^2(j)$	$Pcc(j)$
1	0	$(1,1)$	0	$(1,1)$
2	7	$(1,2)$	7	$(1,2)$
3	8	$(1,3)$	8	$(1,3)$
4	$+\infty$	$(1,4)$	11	$(1,2,4)$
5	$+\infty$	$(1,5)$	8	$(1,2,5)$
6	$+\infty$	$(1,6)$	9	$(1,2,6)$

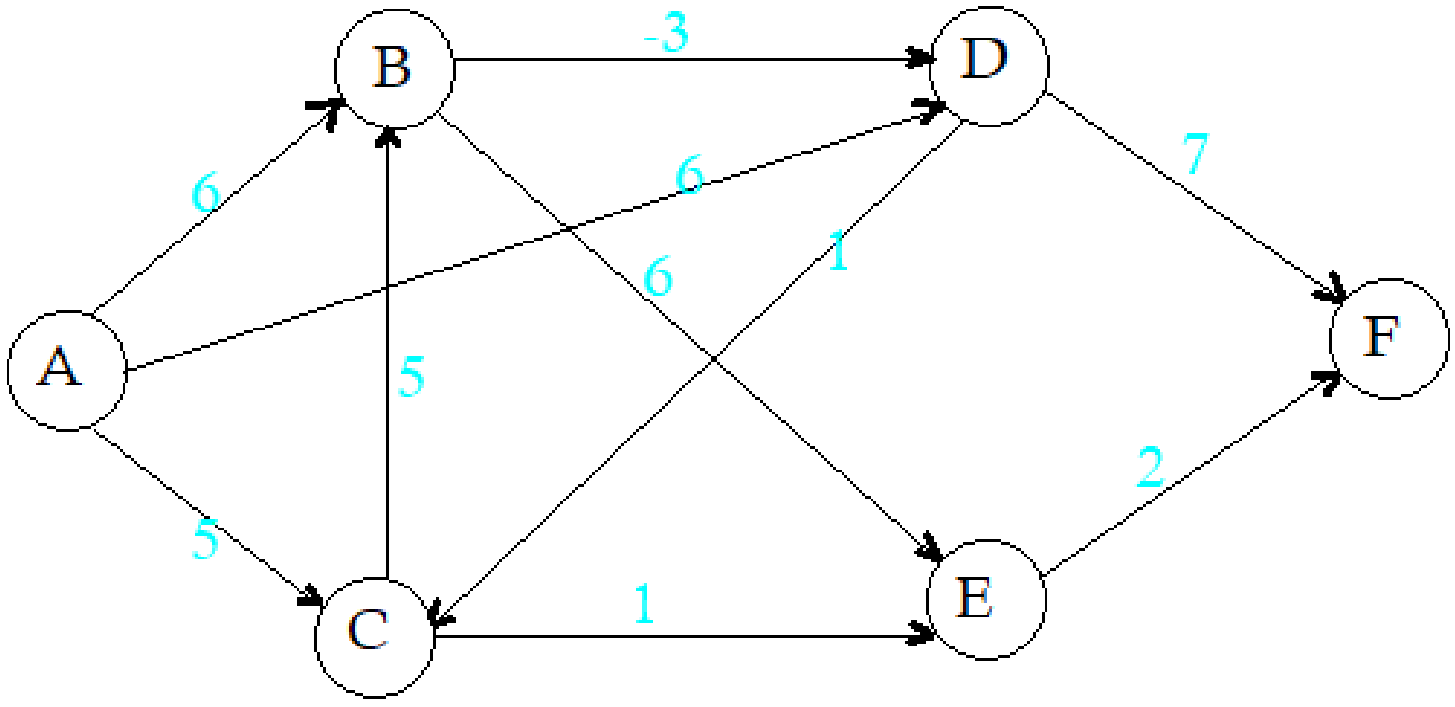
	$k=1$		$k=2$		$k=3$	
	$d^1(j)$	$Pcc(j)$	$d^2(j)$	$Pcc(j)$	$d^3(j)$	$Pcc(j)$
1	0	$(1,1)$	0	$(1,1)$	0	$(1,1)$
2	7	$(1,2)$	7	$(1,2)$	7	$(1,2)$
3	8	$(1,3)$	8	$(1,3)$	5	$(1,2,5,3)$
4	$+\infty$	$(1,4)$	11	$(1,2,4)$	10	$(1,2,5,4)$
5	$+\infty$	$(1,5)$	8	$(1,2,5)$	8	$(1,2,5)$
6	$+\infty$	$(1,6)$	9	$(1,2,6)$	9	$(1,2,6)$

	$k=1$		$k=2$		$k=3$		$k=4$	
	$d^1(j)$	$Pcc(j)$	$d^2(j)$	$Pcc(j)$	$d^3(j)$	$Pcc(j)$	$d^4(j)$	$Pcc(j)$
1	0	$(1,1)$	0	$(1,1)$	0	$(1,1)$	0	$(1,1)$
2	7	$(1,2)$	7	$(1,2)$	7	$(1,2)$	7	$(1,2)$
3	8	$(1,3)$	8	$(1,3)$	5	$(1,2,5,3)$	5	$(1,2,5,3)$
4	$+\infty$	$(1,4)$	11	$(1,2,4)$	10	$(1,2,5,4)$	10	$(1,2,5,4)$
5	$+\infty$	$(1,5)$	8	$(1,2,5)$	8	$(1,2,5)$	8	$(1,2,5)$
6	$+\infty$	$(1,6)$	9	$(1,2,6)$	9	$(1,2,6)$	7	$(1,2,5,3,6)$

	$k=1$		$k=2$		$k=3$		$k=4$		$k=5$	
	$d^1(j)$	$Pcc(j)$	$d^2(j)$	$Pcc(j)$	$d^3(j)$	$Pcc(j)$	$d^4(j)$	$Pcc(j)$	$d^5(j)$	$Pcc(j)$
1	0	$(1,1)$	0	$(1,1)$	0	$(1,1)$	0	$(1,1)$	0	$(1,1)$
2	7	$(1,2)$	7	$(1,2)$	7	$(1,2)$	7	$(1,2)$	7	$(1,2)$
3	8	$(1,3)$	8	$(1,3)$	5	$(1,2,5,3)$	5	$(1,2,5,3)$	5	$(1,2,5,3)$
4	$+\infty$	$(1,4)$	11	$(1,2,4)$	10	$(1,2,5,4)$	10	$(1,2,5,4)$	10	$(1,2,5,4)$
5	$+\infty$	$(1,5)$	8	$(1,2,5)$	8	$(1,2,5)$	8	$(1,2,5)$	8	$(1,2,5)$
6	$+\infty$	$(1,6)$	9	$(1,2,6)$	9	$(1,2,6)$	7	$(1,2,5,3,6)$	7	$(1,2,5,3,6)$

Exemple 2.

Appliquer l'algorithme de Ford-Bellman pour trouver le chemin minimal entre *A* et *F*.



	$k=1$	
	$d^1(j)$	$Pcc(j)$
A	0	(A,A)
B	6	(A,B)
C	5	(A,C)
D	6	(A,D)
E	$+\infty$	(A,E)
F	$+\infty$	(A,F)

Algorithme de Floyd

- l'algorithme de Floyd-Warshall est un algorithme pour déterminer les distances des plus courts chemins entre toutes les paires de sommets dans un graphe orienté et pondéré.
- L'algorithme de Floyd autorise la présence des arcs de poids négatif.
- En plus de la recherche du plus court chemin, l'algorithme permet la détection de circuits de tailles négatives.

Algorithme de Floyd: Algorithme

- Etape 0: Initialisation

  Faire:

  $\forall i \in \{1, ..., n\}$, $d(i, i) = 0$

  $\forall i, j$ tq. $i \neq j$

  $$\text{d(i,j)} = \begin{cases} C_{i,j} & \text{si l'arc(i,j)} \in \text{graphe} \\ \infty & \text{si non} \end{cases}$$

  $$PCC(i, j) = (i, j)$$

  $k = 1$

- Etape 1: Mise à jour

  Faire:

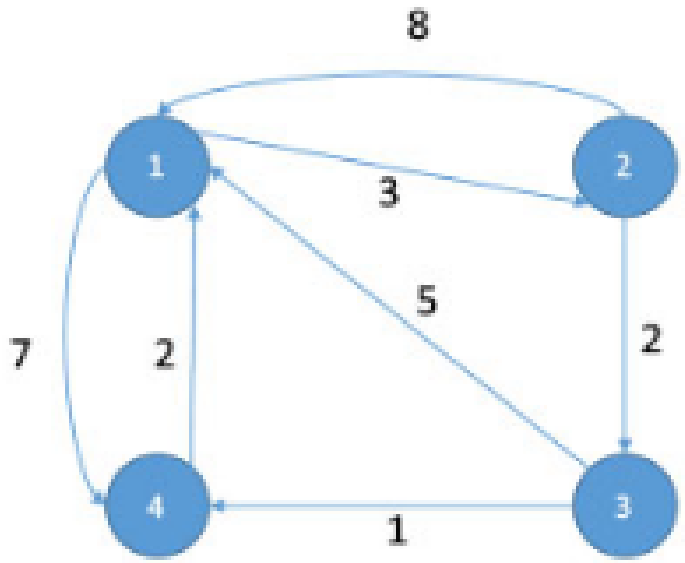
  $\forall i, j \neq k$, si $d(i, k) + d(k, j) < d(i, j)$

  Alors $d(i, k) + d(k, j) = d(i, j)$ et $PCC(i, j) = PCC(i, k) \cup PCC(k, j)$

  Fin.

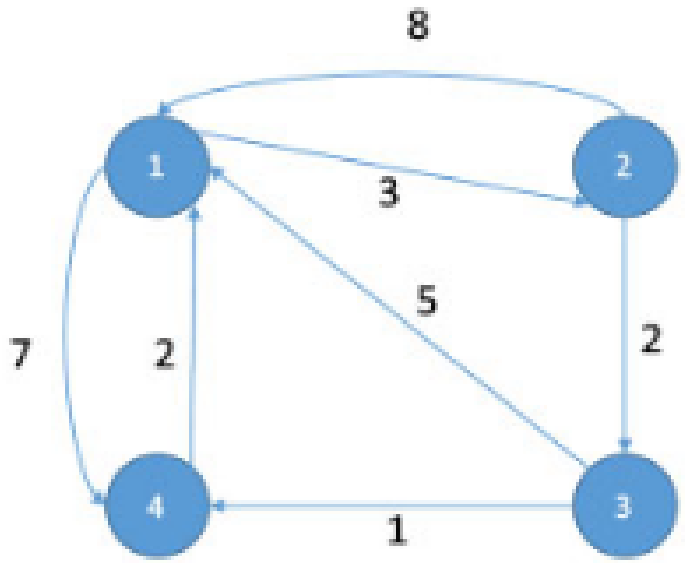
- Etape 2: Test de convergence
  S'il existe i tel que $d(i,i) < 0$ alors Fin (absence d'un plus court chemin).
  Sinon si $k = n$ alors Fin (optimalité).
  Sinon $k = k + 1$ et aller à l'étape 1.

Exemple



$$A^0 = \begin{pmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{pmatrix}$$

Exemple



$$A^0 = \begin{pmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & \infty \\ 5 & \infty & 0 & 1 \\ 2 & \infty & \infty & 0 \end{pmatrix}$$

Considérer le sommet 1

$$A^1 = \begin{pmatrix} 0 & 3 & \infty & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & \infty & 0 \end{pmatrix}$$

Considérer le sommet 2

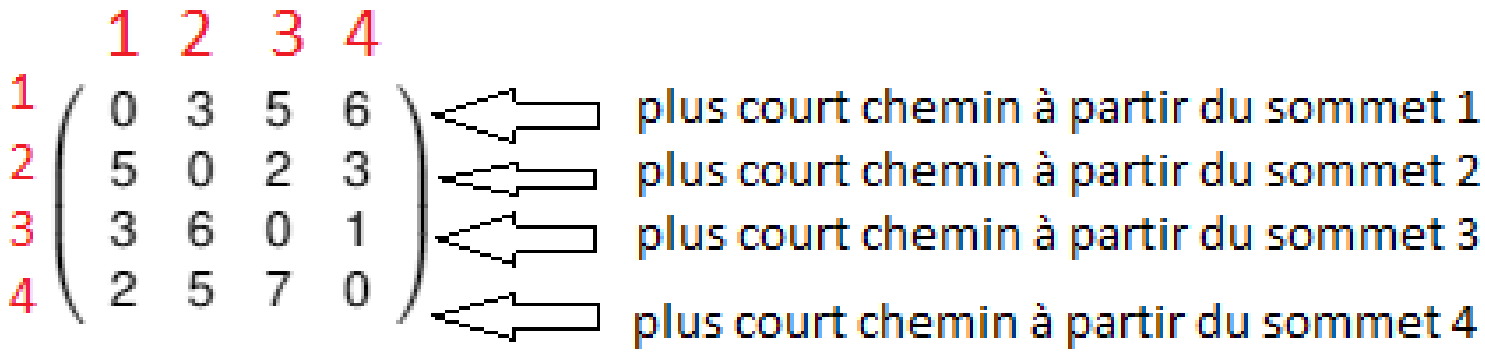
$$A^2 = \begin{pmatrix} 0 & 3 & 5 & 7 \\ 8 & 0 & 2 & 15 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{pmatrix}$$

Considérer le sommet 3

$$A^3 = \begin{pmatrix} 0 & 3 & 5 & 6 \\ 7 & 0 & 2 & 3 \\ 5 & 8 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{pmatrix}$$

Considérer le sommet 4

$$A^4 = \begin{pmatrix} 0 & 3 & 5 & 6 \\ 5 & 0 & 2 & 3 \\ 3 & 6 & 0 & 1 \\ 2 & 5 & 7 & 0 \end{pmatrix}$$


1 2 3 4
1 0 3 5 6
2 5 0 2 3
3 3 6 0 1
4 2 5 7 0
plus court chemin à partir du sommet 1
plus court chemin à partir du sommet 2
plus court chemin à partir du sommet 3
plus court chemin à partir du sommet 4

Application 1: Affectation du personnel d'une compagnie de transport

Une compagnie de transport urbain décide de faire appel à des conducteurs de bus supplémentaires. Pour cela, elle doit préciser les horaires de travail de chaque conducteur. Le tableau suivant présente les différents créneaux horaires possibles ainsi que leurs coûts. L'objectif est d'avoir, pour chaque heure de la période de service (de 9*h* à 17*h*), au moins un conducteur en service et ce avec un coût minimal.

Horaires	9-13	9-11	12-15	12-17	14-17	13-16	16-17
Coût	30	18	21	38	20	22	9

Résoudre ce problème en le formulant comme un problème de plus court chemin dans un graphe qu'on précisera.

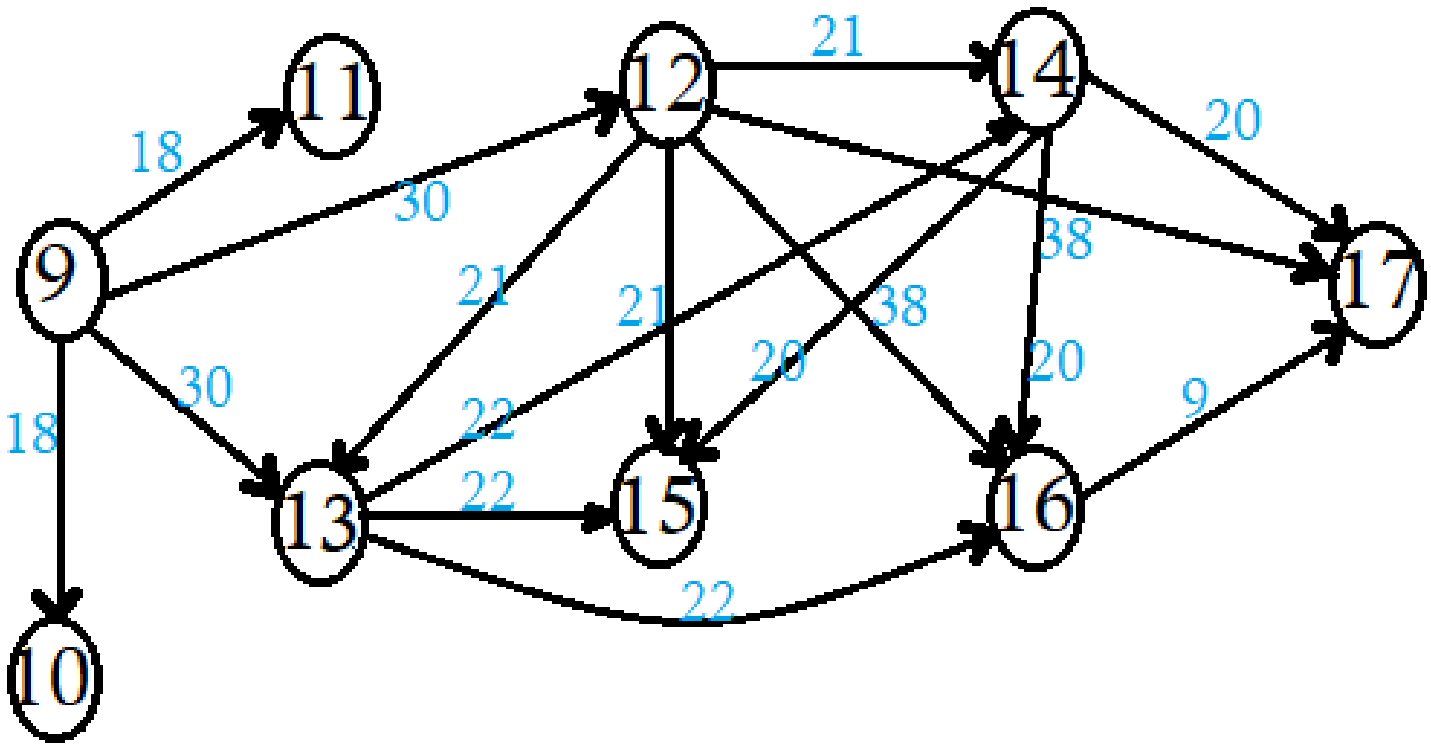
Correction:

On définit le graphe G de la manière suivante:

• un sommet est associé à chaque début d'heure,

• un arc reliant les sommets i et j $(i < j)$ décrit une periode de travail qui commence à l'heure (i) et qui s'achève à l'heure (j).

• La longueur de chaque arc est le coût associé au créneau horaire correspondant.



- Sommet 9:

$C_{9,10} = C_{9,11} = 18$

$C_{9,12} = C_{9,13} = 30$

- Sommet 12:

$C_{12,13} = C_{12,14} = C_{12,15} = 21$

$C_{12,16} = C_{12,17} = 38$

- Sommet 13:

$C_{13,14} = C_{13,15} = C_{13,16} = 22$

- Sommet 14:

$C_{14,15} = C_{14,16} = C_{14,17} = 20$

- Sommet 16:

$C_{16,17} = 9$

La solution optimale est $(9, 13, 16, 17)$ avec un coût minimal de 61.
Les créneaux horaires retenus sont: $[9 - 13]$, $[13 - 16]$ et $[16 - 17]$.